ARCHITEKTUR & MODERNES ENGINEERING

Technologie, Tooling & Cloud-Native Workflows

Eine Übersicht über unterstützte Sprachen, Frameworks, CI/CD-Pipelines und Kubernetes Best Practices.

Programmiersprachen & Runtimes

Effiziente Entwicklung von Data Pipelines bis zu High-Performance Systemen.

Python

Data Science, FastAPI & Ausführung im Interpreter.

Go (Golang)

Hochparallele Microservices & schnelle CLI-Tools.

Rust

Maximale Speichersicherheit ohne Garbage Collection.

JS / TS

Moderne Web-Apps mit Node.js, Bun und React.

Systemnahe Entwicklung & Skripting

C / C++ Infrastructure

POSIX & Win32: Low-Level Systemprogrammierung.

Build-Systeme: Automatisiert mit CMake & Makefiles.

Analyse: Speicheranalyse via Valgrind.

Shell & Automatisierung

Bash: ShellCheck-validierte Pipelineskripte.

PowerShell: Cross-Platform Admin-Skripte.

Textverarbeitung: Effiziente Nutzung von Sed, Awk & Grep.

DevOps & Infrastructure as Code

Moderne Cloud-Infrastruktur durch declarative Konfigurationen.



Docker

Multi-Stage Minimal Builds



Kubernetes

Gehärtete Deployments & Helm



Terraform

HCL Multi-Cloud Provisionierung

GitHub Actions Pipeline Architektur

PHASE 1

Lint & Format

Strikte Code-Formatierung via Clippy & Golangci-lint.

→

PHASE 2

Test & Coverage

Automatische Race-Detection und Unit-Tests.

→

PHASE 3

Build & Scan

Multi-Stage Container-Erstellung & Vulnerability Check.

→

PHASE 4

K8s Deploy

Rolling Update mit Health-Check Rollback.

Docker Production Best Practices

Sicherheitsrichtlinien für gehärtete Container-Images.

- **Minimal Base Images:** Nutze *scratch* für Go und *distroless* für Rust.
- **Non-Root Privilege:** Zuweisung expliziter UIDs (`USER 10001`).
- **Multi-Stage Separation:** Strikte Trennung von Build-Tools und Laufzeit-Binarys.
- **Binary Optimization:** Dead-Code-Elimination und Stripping von Debug-Symbolen.

Kubernetes Workload Härtung

⚠ Unsicher (Default)

- Root-Ausführung im Container
- Beschreibbares Dateisystem
- Unbegrenzte CPU/RAM Ressourcennutzung
- Fehlende Health-Probes

✓ Gehärtet (Production)

- `runAsNonRoot: true` erzwungen
- `readOnlyRootFilesystem: true`
- Exakte CPU & Memory Limits
- Konfigurierte Liveness/Readiness Probes

Git & Qualitätssicherung

Conventional Commits
Strukturierte History für Changelog-Automatisierung.

Automatisierte Unit-Tests
Integration in Pre-Commit Hooks und CI.

Branch Protection
Merges nur nach erfolgreicher CI-Check-Pipeline.

Static Code Analysis
Automatische Erkennung von Sicherheitslücken.

Performance Impact

< 15 MB

Durchschnittliche Container-Größe für kompilierte Go/Rust Binaries mit Multi-Stage Scratch-Builds.

Vielen Dank!

Bereit für die Umsetzung moderner CI/CD-Pipelines und Cloud-Native Workflows?

Fragen & Diskussion